

EXPERIMENT 9

Introduction to Flask — Building a Basic Web Application

Aim

To set up a basic Flask web application, define routes, render templates, and understand the request–response cycle of a WSGI web framework.

Theory

Flask is a lightweight 'micro' web framework for Python built on Werkzeug (WSGI) and Jinja2 (templating). 'Micro' means it ships with a minimal core and leaves choices (database, forms, auth) to the developer via extensions, making it ideal for APIs, prototypes and ML model serving.

A Flask application maps URLs to Python functions called view functions using the `@app.route` decorator. When a request arrives, Flask matches the URL to a route, runs the view function, and returns its return value as the HTTP response. This is the core request–response cycle.

Dynamic HTML is produced with Jinja2 templates stored in a 'templates' folder, where `{{ }}` injects variables and `{% %}` controls logic. Static assets (CSS/JS) live in a 'static' folder. Debug mode enables auto-reload and an interactive traceback.

Software / Tools Required

Python 3.10+, Flask (pip install flask), a browser.

Procedure (Step-by-Step)

Step 1. Install Flask inside your virtual environment.

```
pip install flask
```

Step 2. Create the minimal application with a single route.

```
# app.py
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello, Flask! Welcome to U12VS501."

if __name__ == "__main__":
    app.run(debug=True)
```

Step 3. Run the development server and open the URL shown (<http://127.0.0.1:5000>).

```
python app.py
```

Step 4. Add a dynamic route that captures a URL parameter.

```
@app.route("/user/<name>")
def greet(name):
    return f"Hello, {name}!"
```

Step 5. Add a route with a typed converter and return an HTML string.

```
@app.route("/square/<int:n>")
def square(n):
    return f"<h2>{n} squared is {n*n}</h2>"
```

Step 6. Create a Jinja2 template for proper HTML rendering.

```
<!-- templates/index.html -->
```

```
<html><body>
  <h1>Welcome {{ user }}</h1>
  <ul>{% for c in courses %}<li>{{ c }}</li>{% endfor %}</ul>
</body></html>
```

Step 7. Render the template from a view function, passing data.

```
from flask import render_template
@app.route("/dashboard")
def dashboard():
    return render_template("index.html", user="Student",
                           courses=["Git", "Flask", "Streamlit"])
```

Step 8. Serve a static CSS file from the static/ folder.

```
# static/style.css
body { font-family: sans-serif; background: #f5f5f5; }
<!-- in template head: -->
<link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
```

Step 9. Inspect the request object (query string, method).

```
from flask import request
@app.route("/search")
def search():
    q = request.args.get("q", "")
    return f"You searched for: {q}" # /search?q=flask
```

Step 10. Set a custom 404 error handler.

```
@app.errorhandler(404)
def not_found(e):
    return "<h3>Page not found</h3>", 404
```

Expected Output

Visiting '/' shows the greeting; '/user/Student' returns a personalised message; '/dashboard' renders a styled HTML page with the course list from the template — confirming routing and templating work.

Conclusion

A basic Flask web application with static and dynamic routes, Jinja2 templating and static assets was built, establishing the foundation for serving ML applications over HTTP.

Viva Questions

1. What does the @app.route decorator do?
2. What is WSGI and where does Flask use it?
3. What is the difference between the static and templates folders?
4. Explain the request–response cycle in Flask.
5. Why should debug mode be disabled in production?